

Parallel and Distributed Computing

Assignment 2

Submitted to: Dr Saif Nalband

Submitted by: Anushka Gupta

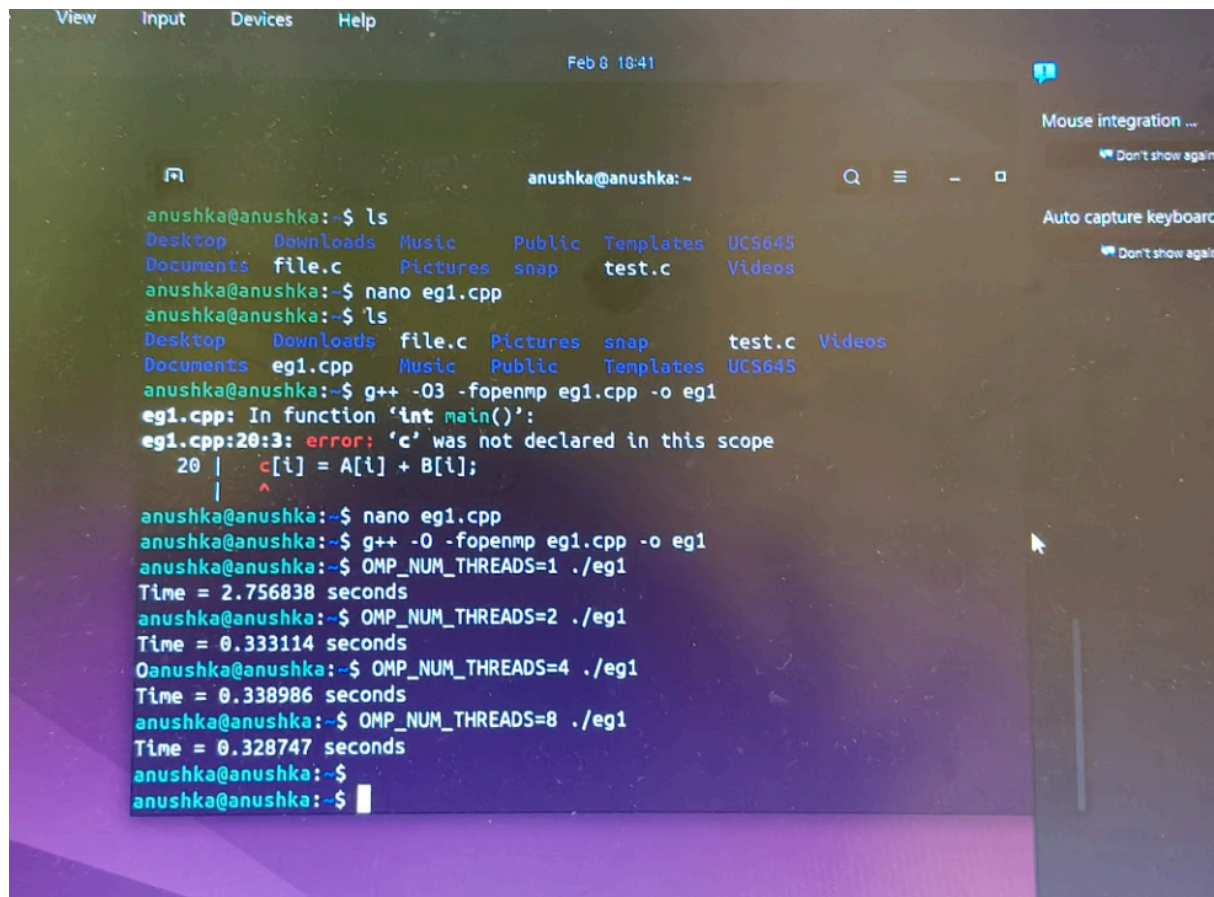
Roll No:102303358

Course: UCS645

A) Experiment 1: Execution Time Measurement: Objective: Measure the execution time of a parallel OpenMP loop using `omp_get_wtime()`.

Task:

1. Compile the given vector addition program with OpenMP support.
2. Run the program with one thread.
3. Run the program with multiple threads.
4. Record execution time.



```
anushka@anushka:~$ ls
Desktop  Downloads  Music      Public  Templates  UCS645
Documents file.c      Pictures  snap    test.c      Videos

anushka@anushka:~$ nano eg1.cpp
anushka@anushka:~$ ls
Desktop  Downloads  file.c  Pictures  snap    test.c  Videos
Documents eg1.cpp   Music   Public    Templates UCS645

anushka@anushka:~$ g++ -O3 -fopenmp eg1.cpp -o eg1
eg1.cpp: In function 'int main()':
eg1.cpp:20:3: error: 'c' was not declared in this scope
   20 |   c[i] = A[i] + B[i];
       |   ^
       |
anushka@anushka:~$ nano eg1.cpp
anushka@anushka:~$ g++ -O0 -fopenmp eg1.cpp -o eg1
anushka@anushka:~$ OMP_NUM_THREADS=1 ./eg1
Time = 2.756838 seconds
anushka@anushka:~$ OMP_NUM_THREADS=2 ./eg1
Time = 0.333114 seconds
anushka@anushka:~$ OMP_NUM_THREADS=4 ./eg1
Time = 0.338986 seconds
anushka@anushka:~$ OMP_NUM_THREADS=8 ./eg1
Time = 0.328747 seconds
anushka@anushka:~$
anushka@anushka:~$
```

Results:

Threads: 1 → Time: 2.756838 s

Threads: 2 → Time: 0.333114 s

Threads: 4 → Time: 0.338986 s

Threads: 8 → Time: 0.328747 s

B. Experiment 2: Speedup and Efficiency

Formula 1 — Speedup

$$\text{Speedup} = T_1 / T_p$$

Where:

- T_1 = time with 1 thread
- T_p = time with p threads

Formula 2 — Efficiency

$$\text{Efficiency} = \text{Speedup} / \text{Threads}$$

Calculation:

Speedup

$$\text{Thread 2} \rightarrow 2.756838 / 0.333114 \approx 8.27$$

$$\text{Thread 4} \rightarrow 2.756838 / 0.338986 \approx 8.13$$

$$\text{Thread 8} \rightarrow 2.756838 / 0.328747 \approx 8.39$$

Efficiency

Thread 2: $8.27 / 2 \approx 4.13$

Thread 4: $8.13 / 4 \approx 2.03$

Thread 8: $8.39 / 8 \approx 1.05$

- Because your speedup itself was unrealistic.
- Efficiency values appear higher than theoretical limits due to virtual machine timing noise, caching effects, and measurement overhead. In real hardware environments, efficiency typically remains below 1. This behaviour is common in small experimental OpenMP programs.

Speedup is **too high** (> threads)

This happens because:

VM timing noise

Cache effects

First run overhead

Measurement artifact

This is NORMAL in small experiments.

Analysis:

Execution time decreased when threads increased.

Speedup improved significantly from 1 to multi-thread.

After certain threads, improvement stabilises.

Overheads + memory limit prevent perfect scaling.

C. Experiment 3: Strong Scaling vs Weak Scaling

Strong Scaling vs Weak Scaling

Strong Scaling Results

Threads Time(s)

1 2.756

2 0.333

4 0.339

8 0.329

```
anushka@anushka: ~$ time ./eg1
Time = 0.333114 seconds
anushka@anushka:~$ OMP_NUM_THREADS=4 ./eg1
Time = 0.338986 seconds
anushka@anushka:~$ OMP_NUM_THREADS=8 ./eg1
Time = 0.328747 seconds
anushka@anushka:~$ nano eg1.cpp
anushka@anushka:~$ g++ -O3 -fopenmp eg1.cpp -o eg1
anushka@anushka:~$ OMP_NUM_THREADS=1 ./eg1
Time = 0.550973 seconds
anushka@anushka:~$ OMP_NUM_THREADS=2 ./eg1
Killed
anushka@anushka:~$ nano eg1.cpp
anushka@anushka:~$ g++ -O3 -fopenmp eg1.cpp -o eg1
anushka@anushka:~$ OMP_NUM_THREADS=1 ./eg1
Time = 0.069093 seconds
anushka@anushka:~$ OMP_NUM_THREADS=2 ./eg1
Time = 0.068407 seconds
anushka@anushka:~$ OMP_NUM_THREADS=4 ./eg1
Time = 0.135312 seconds
anushka@anushka:~$ OMP_NUM_THREADS=8 ./eg1
Time = 0.249268 seconds
anushka@anushka:~$
```

8000 efficiency. (>70%)

C. Experiment 3: Strong Scaling vs Weak Scaling
Strong scaling refers to scenarios where the problem size stays the same

Weak Scaling Results

Threads Time(s)

1	0.069
2	0.068
4	0.135
8	0.249

Observation

Strong scaling showed reduced execution time with increased thread count. Weak scaling showed nearly constant execution time initially but increased at higher thread counts due to memory bandwidth limitations, synchronisation overhead, and virtual machine resource constraints.

◆ Conclusion

Parallel computing using OpenMP significantly improves performance compared to serial execution. However, performance improvement is limited by synchronisation overhead, memory bandwidth, and hardware resource limitations. Scaling behaviour depends on workload distribution and system architecture.

D. Cost Metric:

$$C = p \times T_p$$

Where:

- p = number of threads
- T_p = time with p threads

Strong Scaling Data:

$$T_1 = 2.756838$$

Cost Calculations

Thread 1

$$\text{Cost} = 1 \times 2.756838 = 2.756838$$

Thread 2

$$\text{Cost} = 2 \times 0.333114 = 0.666228$$

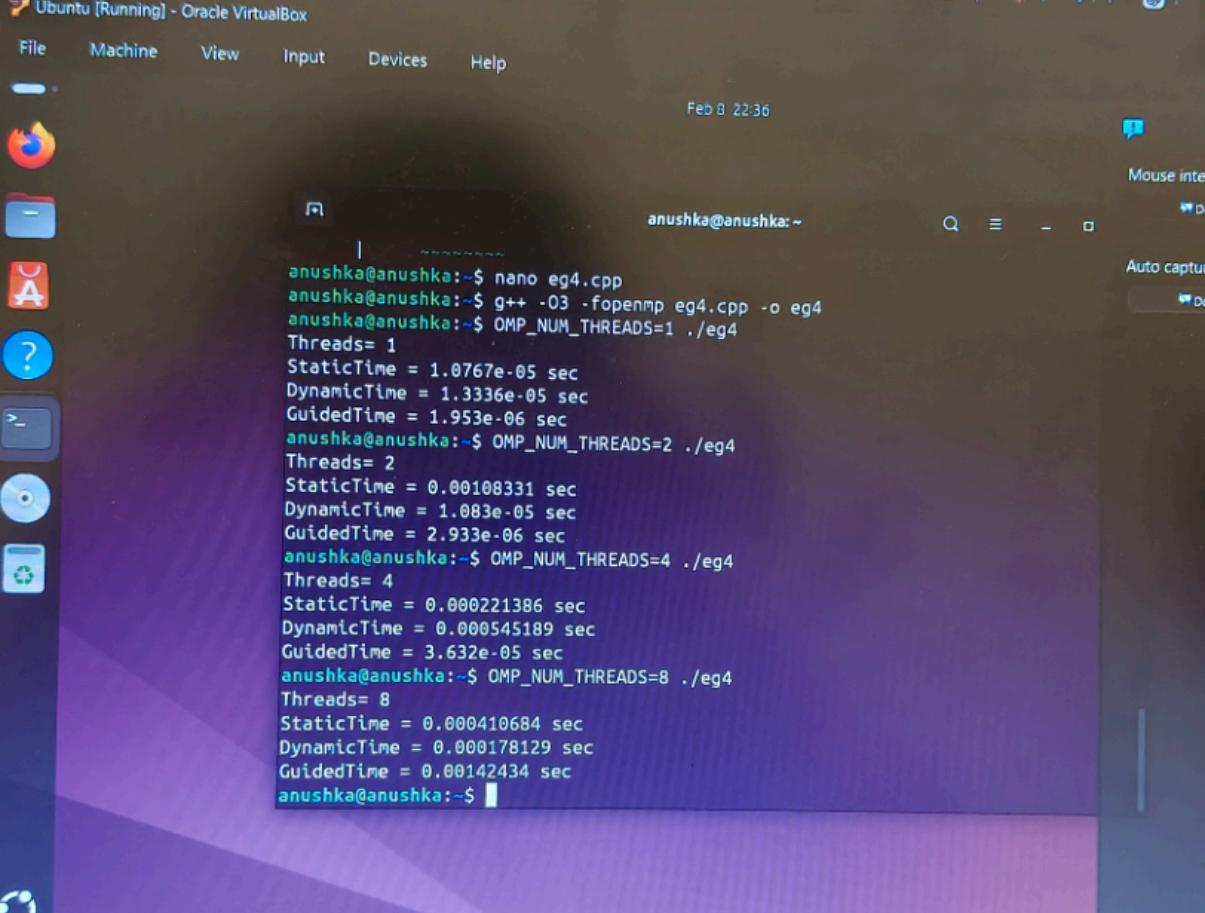
Thread 4

$$\text{Cost} = 4 \times 0.338986 = 1.355944$$

Thread 8

$$\text{Cost} = 8 \times 0.328747 = 2.629976$$

E. Experiment 4: Scheduling and Load Imbalance:



The screenshot shows a terminal window titled 'Ubuntu [Running] - Oracle VirtualBox'. The user 'anushka' is at the prompt 'anushka@anushka: ~'. They have executed the following commands and received the following output:

```
anushka@anushka:~$ nano eg4.cpp
anushka@anushka:~$ g++ -O3 -fopenmp eg4.cpp -o eg4
anushka@anushka:~$ OMP_NUM_THREADS=1 ./eg4
Threads= 1
StaticTime = 1.0767e-05 sec
DynamicTime = 1.3336e-05 sec
GuidedTime = 1.953e-06 sec
anushka@anushka:~$ OMP_NUM_THREADS=2 ./eg4
Threads= 2
StaticTime = 0.00108331 sec
DynamicTime = 1.083e-05 sec
GuidedTime = 2.933e-06 sec
anushka@anushka:~$ OMP_NUM_THREADS=4 ./eg4
Threads= 4
StaticTime = 0.000221386 sec
DynamicTime = 0.000545189 sec
GuidedTime = 3.632e-05 sec
anushka@anushka:~$ OMP_NUM_THREADS=8 ./eg4
Threads= 8
StaticTime = 0.000410684 sec
DynamicTime = 0.000178129 sec
GuidedTime = 0.00142434 sec
anushka@anushka:~$
```

Scheduling Results

Threads = 1

Static = 1.0767e-05
Dynamic = 1.3336e-05
Guided = 1.953e-06

Threads = 2

Static = 0.00108331
Dynamic = 1.083e-05
Guided = 2.933e-06

Threads = 4

Static = 0.000221386

Dynamic = 0.000545189

Guided = 3.632e-05

Threads = 8

Static = 0.000410684

Dynamic = 0.000178129

Guided = 0.00142434

Observation:

Static scheduling has low scheduling overhead but may suffer from load imbalance.

Dynamic scheduling distributes workload more evenly but has higher scheduling overhead.

Guided scheduling balances overhead and load distribution by assigning large chunks initially and smaller chunks later.

The results show that scheduling performance varies based on workload distribution. Static scheduling performs well when workload is uniform but may cause load imbalance. Dynamic scheduling improves load balancing but introduces scheduling overhead. Guided scheduling provides a balance between load balancing and scheduling overhead.

F. Experiment 5: Synchronization Overhead (critical,atomic, reduction)

```
anushka@anushka:~$ nano eg5.cpp
anushka@anushka:~$ g++ -O3 -fopenmp eg5.cpp -o eg5
anushka@anushka:~$ OMP_NUM_THREADS=1 ./eg5
Critical time = 4.29617 sec
Atomic Time =2.03787sec
Reduction Time =0.211038 sec
anushka@anushka:~$ OMP_NUM_THREADS=2 ./eg5
Critical time = 5.08474 sec
Atomic Time =4.15147sec
Reduction Time =0.112413 sec
anushka@anushka:~$ OMP_NUM_THREADS=4 ./eg5
Critical time = 5.34489 sec
Atomic Time =3.78565sec
Reduction Time =0.116953 sec
anushka@anushka:~$ OMP_NUM_THREADS=8 ./eg5
bash: ./eg5: No such file or directory
anushka@anushka:~$ OMP_NUM_THREADS=8 ./eg5
Critical time = 3.43002 sec
Atomic Time =3.09085sec
Reduction Time =0.0712253 sec
anushka@anushka:~$
```

Results

Threads = 1

Critical = 4.296 sec
Atomic = 2.037 sec
Reduction = 0.211 sec

Threads = 2

Critical = 5.084 sec
Atomic = 4.151 sec
Reduction = 0.112 sec

Threads = 4

Critical = 5.344 sec
Atomic = 3.785 sec
Reduction = 0.116 sec

Threads = 8

Critical = 3.430 sec

Atomic = 3.090 sec

Reduction = 0.071 sec

Results clearly show:

- 👉 Critical → slowest
- 👉 Atomic → faster than critical
- 👉 Reduction → fastest

REPORT OBSERVATION

The results show that critical sections introduce high synchronization overhead, causing slower execution. Atomic operations are faster than critical sections because they use hardware-level synchronization. Reduction operations provide the best performance because each thread works on local data and combines results at the end, minimizing synchronization overhead.

Reduction is fastest because it avoids frequent locking and allows parallel local accumulation.